

# Lab5 Challenge: Copy on Write (COW) 设计报告

## 1. 概述

本报告描述了在 ucore 操作系统中实现 Copy on Write（写时复制）机制的设计与实现。

## 2. COW 机制原理

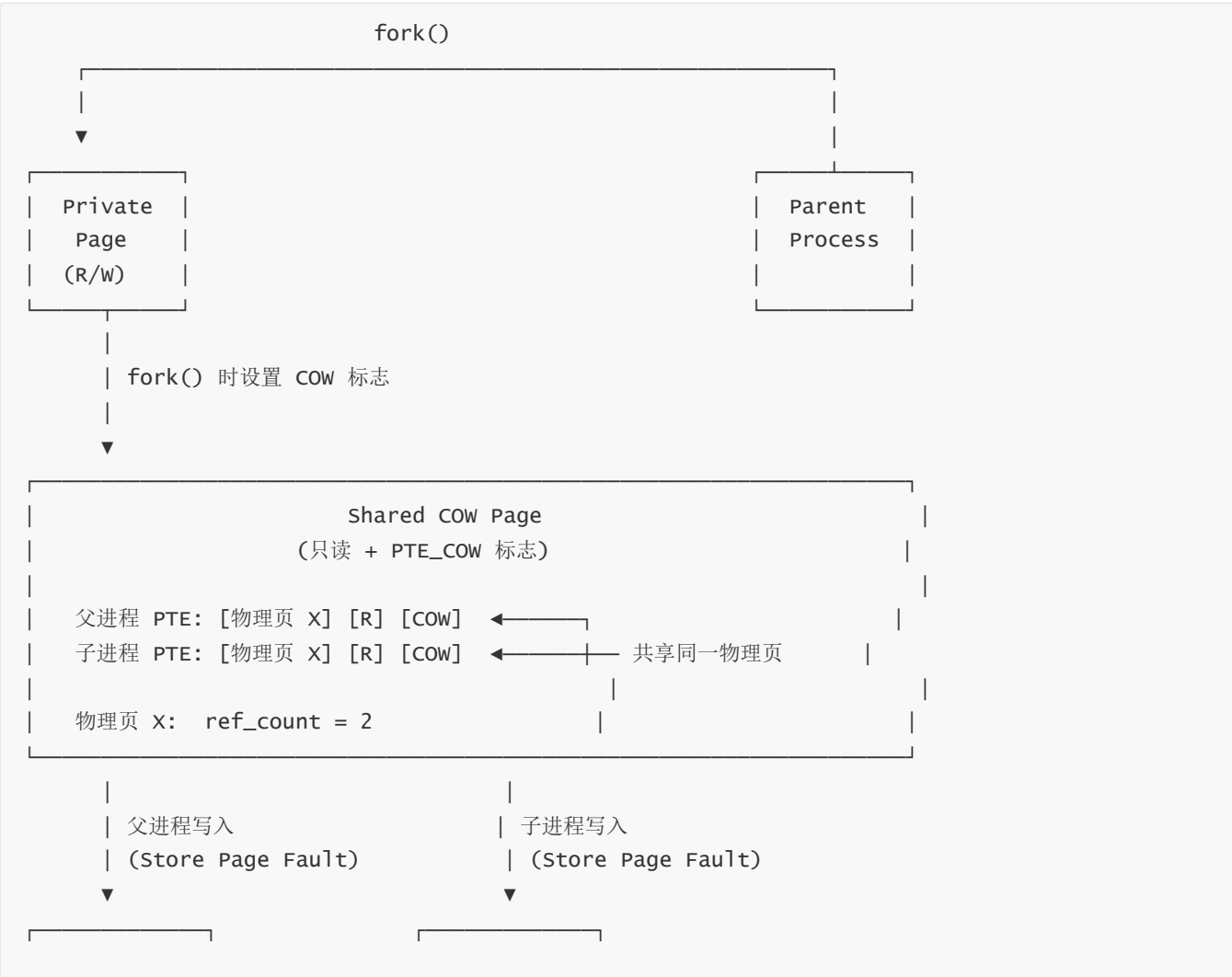
### 2.1 什么是 COW

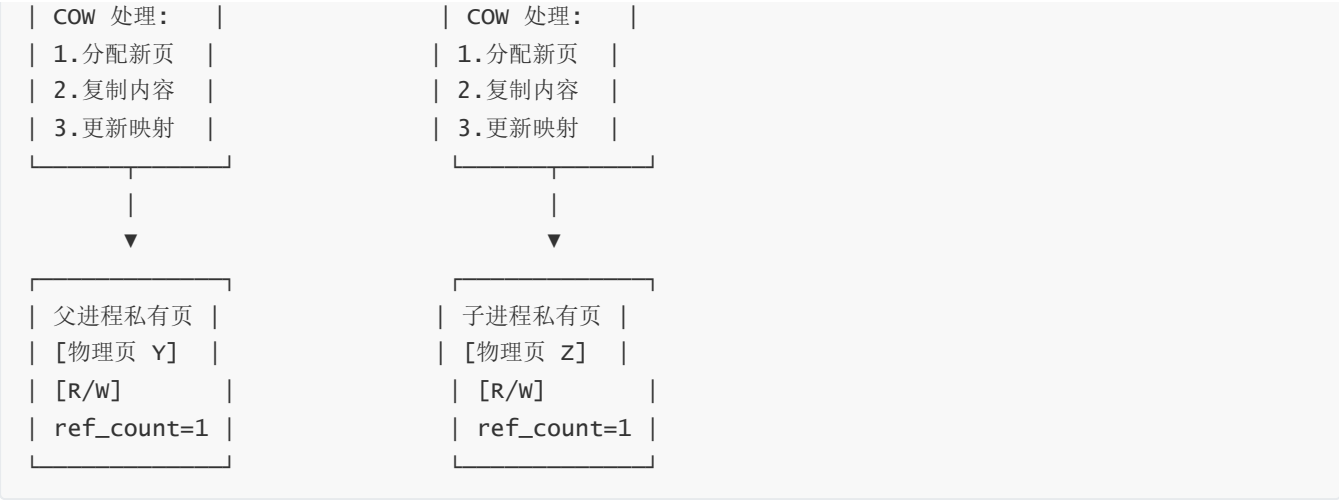
Copy on Write（写时复制）是一种延迟复制的优化策略。当父进程 fork 创建子进程时，不立即复制父进程的内存页面，而是让父子进程共享同一物理页面，并将这些页面标记为只读。只有当某个进程尝试写入这些共享页面时，才会触发页面错误，此时系统才真正复制该页面。

### 2.2 COW 的优点

- 1. 节省内存：fork 后如果子进程立即 exec，则无需复制任何页面
- 2. 提高性能：避免不必要的内存复制操作
- 3. 延迟开销：将复制开销分散到实际需要时

## 3. 状态转换图（有限状态自动机）





3.1 页面状态定义

| 状态         | PTE_W | PTE_COW | ref_count | 描述        |
|------------|-------|---------|-----------|-----------|
| PRIVATE_RW | 1     | 0       | 1         | 私有可写页面    |
| SHARED_COW | 0     | 1       | ≥2        | COW共享只读页面 |
| LAST_COW   | 0     | 1       | 1         | 最后一个COW引用 |

3.2 状态转换事件

| 事件          | 触发条件       | 转换                              |
|-------------|------------|---------------------------------|
| fork        | 进程复制       | PRIVATE_RW → SHARED_COW         |
| write_fault | 写COW页面     | SHARED_COW → PRIVATE_RW (分配新页)  |
| last_write  | 写最后一个COW引用 | LAST_COW → PRIVATE_RW (原地恢复写权限) |
| exit        | 进程退出       | ref_count--                     |

4. 实现细节

4.1 修改的文件

| 文件               | 修改内容                  |
|------------------|-----------------------|
| kern/mm/mmu.h    | 添加 PTE_COW 标志位定义      |
| kern/mm/pmm.c    | 修改 copy_range 支持 COW  |
| kern/mm/vmm.c    | 添加 do_pgfault 处理 COW  |
| kern/mm/vmm.h    | 添加 do_pgfault 声明      |
| kern/trap/trap.c | 修改页面错误处理调用 do_pgfault |

## 4.2 PTE\_COW 标志位

```
// kern/mm/mmu.h
// COW (Copy on Write) 标志位 - 使用软件保留位
#define PTE_COW 0x100 // 写时复制标志 (bit 8, 位于软件保留区域)
```

RISC-V 页表项的 bit 8-9 是软件保留位，我们使用 bit 8 作为 COW 标志。

## 4.3 copy\_range 核心实现

```
// kern/mm/pmm.c
if (share)
{
    // COW实现：父子进程共享同一物理页，都设置为只读+COW标志
    // 1. 将父进程的页面设置为只读，并添加COW标志
    *ptep = (*ptep & ~PTE_W) | PTE_COW;
    // 2. 子进程映射到同一物理页，也设置为只读+COW标志
    perm = (perm & ~PTE_W) | PTE_COW;
    ret = page_insert(to, page, start, perm);
    // 3. 刷新TLB，使父进程的页表修改生效
    tlb_invalidate(from, start);
}
```

## 4.4 do\_pgfault COW 处理

```
// kern/mm/vmm.c
if ((*ptep & PTE_COW) && !(*ptep & PTE_W))
{
    struct Page *old_page = pte2page(*ptep);

    // 优化：如果只有一个引用，直接恢复写权限
    if (page_ref(old_page) == 1)
    {
        *ptep = (*ptep & ~PTE_COW) | PTE_W;
        tlb_invalidate(mm->pgdir, addr);
        return 0;
    }

    // 多个引用：分配新页并复制
    struct Page *new_page = alloc_page();
    memcpy(page2kva(new_page), page2kva(old_page), PGSIZE);
    page_insert(mm->pgdir, new_page, ROUNDOWN(addr, PGSIZE), perm);
}
```

## 5. 测试用例

测试程序 `user/cowtest.c` 验证以下场景：

1. fork 后父子进程共享内存
2. 子进程修改变量后，父进程的值不变

3. 父进程修改变量后，子进程的值不变

4. 大数组跨页面的 COW 正确性

测试结果：

```
COW Test: 开始测试写时复制机制
[子进程] 修改后: global_var = 999
[父进程] 子进程修改后读取: global_var = 100 (应为100)
[父进程] COW测试通过!
[子进程] COW测试通过!
cowtest pass.
```

## 6. Dirty COW 漏洞深度分析 (CVE-2016-5195)

### 6.1 漏洞概述

Dirty COW（脏牛）是 2016 年由 Phil Oester 发现的 Linux 内核提权漏洞，CVE 编号为 CVE-2016-5195。该漏洞自 Linux 2.6.22（2007年）就已存在，直到 2016 年 10 月 18 日才被修复，影响了几乎所有 Linux 发行版长达 9 年之久。

漏洞危害：

- 非特权本地用户可以获得对只读内存映射的写访问权限
- 攻击者可以修改磁盘上的二进制文件，绕过标准权限机制
- 可用于本地提权，获取 root 权限

### 6.2 漏洞原理详解

漏洞出现在 Linux 内核 `get_user_pages()` 函数处理 COW 页面时的竞态条件（Race Condition）。

Linux 内核中的关键代码路径：

```
faultin_page
handle_mm_fault
__handle_mm_fault
handle_pte_fault
do_fault <- pte 不存在
do_cow_fault <- FAULT_FLAG_WRITE
alloc_set_pte
maybe_mkwwrite(pte_mkdirty(entry), vma) <- 标记页面为脏，但保持只读

# 返回 0 并重试
follow_page_mask
follow_page_pte
(flags & FOLL_WRITE) && !pte_write(pte) <- 重试 fault

faultin_page
handle_mm_fault
__handle_mm_fault
handle_pte_fault
FAULT_FLAG_WRITE && !pte_write
```

```

do_wp_page
    PageAnon() <- 这已经是 COW 页面
    reuse_swap_page <- 页面是我们独占的
    wp_page_reuse
        maybe_mkwrite <- 脏但仍然只读
        ret = VM_FAULT_WRITE

((ret & VM_FAULT_WRITE) && !(vma->vm_flags & VM_WRITE)) <- 我们丢弃 FOLL_WRITE

# 返回 0 并作为读错误重试
cond_resched -> 不同的线程现在通过 madvise 取消映射 <-- 竞态条件发生点!

follow_page_mask
    !pte_present && pte_none

faultin_page
    handle_mm_fault
        __handle_mm_fault
            handle_pte_fault
                do_fault <- pte 不存在
                do_read_fault <- 这是读错误, 我们会得到 pagecache 页面!

```

## 攻击原理图解:

### Dirty COW 竞态条件攻击

时间轴 

| 线程 A (madvise 线程)         | 线程 B (写入线程, 通过 /proc/self/mem) |
|---------------------------|--------------------------------|
|                           | 1. 打开只读文件并 mmap                |
|                           | 2. 尝试通过 /proc/self/mem 写入      |
|                           | 3. 触发 COW 页面错误                 |
|                           | 4. 内核开始处理 COW                  |
|                           | - 分配新页面                        |
|                           | - 复制内容                         |
|                           | - 设置 VM_FAULT_WRITE            |
|                           | - 丢弃 FOLL_WRITE 标志             |
| 5. madvise(MADV_DONTNEED) | ← 竞态窗口!                        |
| 告诉内核"我不需要这个页面"            |                                |
| 内核清除页表项                   |                                |
|                           | 6. 内核继续处理 (作为读操作)              |
|                           | - 发现页面不存在                      |
|                           | - 执行 do_read_fault             |
|                           | - 获取原始的 pagecache 页面           |
|                           | 7. 写入操作写到了原始只读文件!              |

结果: 攻击者成功修改了本应只读的文件 (如 /etc/passwd, SUID 程序等)

攻击的两种主要方式：

- 1. 通过 `/proc/self/mem` 写入：利用 `/proc/self/mem` 可以写入进程自己的内存空间
- 2. 通过 `ptrace(PTRACE_POKEDATA)`： `ptrace` 可以写入只读映射

6.3 ucore 中能否模拟此漏洞？

结论：无法在当前 ucore 中模拟此漏洞。

原因分析：

| 漏洞必要条件                      | Linux | ucore | 说明                                 |
|-----------------------------|-------|-------|------------------------------------|
| 多线程支持                       | ✓     | X     | Dirty COW 需要两个线程同时竞争               |
| madvise 系统调用                | ✓     | X     | 需要 <code>MADV_DONTNEED</code> 清除页表 |
| <code>/proc/self/mem</code> | ✓     | X     | 攻击者写入内存的途径                         |
| <code>ptrace</code> 系统调用    | ✓     | X     | 另一种写入只读映射的方式                       |
| 复杂的页面错误处理                   | ✓     | X     | Linux 的多阶段页面错误处理                   |
| 文件内存映射 (mmap)               | ✓     | X     | 需要将文件映射到内存                         |

详细分析：

- 1. 单进程单线程模型
  - ucore 当前每个进程只有一个执行流
  - Dirty COW 需要两个线程同时执行：一个做 `madvise`，一个做 `write`
  - 没有多线程就无法产生必要的竞态条件
- 2. 缺少关键系统调用

```
// Linux 中攻击需要的系统调用，ucore 都没有实现：
madvise(addr, len, MADV_DONTNEED); // 告诉内核丢弃页面
open("/proc/self/mem", O_RDWR);    // 访问进程内存
ptrace(PTRACE_POKEDATA, ...);      // 直接写入内存
mmap(file, PROT_READ, MAP_PRIVATE); // 文件私有映射
```

- 3. 简化的 COW 实现
  - ucore 的 COW 实现是单阶段的：检测 COW 标志 → 复制页面 → 完成
  - Linux 的实现是多阶段的，有多次重试逻辑，竞态窗口在重试之间
  - ucore 没有 `FOLL_WRITE` 标志的丢弃逻辑，这是漏洞的核心
- 4. 页面错误处理的原子性
  - ucore 的 `do_pgfault` 是一个连续的操作，中间没有调度点
  - Linux 在处理过程中有 `cond_resched()`，允许其他线程介入

## 6.4 如何在 ucore 中理论性模拟（教学目的）

虽然无法完整模拟 Dirty COW，但我们可以创建一个简化的概念验证来理解竞态条件：

假设 ucore 支持多线程，模拟代码如下：

```
// 假设的漏洞模拟代码（仅用于教学理解，ucore 实际不支持）

// 有漏洞的 do_pgfault 实现（错误示范）
int do_pgfault_vulnerable(struct mm_struct *mm, uint32_t error_code, uintptr_t addr)
{
    pte_t *ptep = get_pte(mm->pgdir, addr, 0);

    // 阶段1: 检查是否是 COW 页面
    if ((*ptep & PTE_COW) && !(*ptep & PTE_W))
    {
        struct Page *old_page = pte2page(*ptep);
        struct Page *new_page = alloc_page();
        memcpy(page2kva(new_page), page2kva(old_page), PGSIZE);

        // !!! 竞态窗口：如果此时另一个线程调用了"丢弃页面"的操作
        // 并且内核错误地将后续操作视为"读操作"
        // schedule(); // 假设这里发生了调度

        // 阶段2: 如果 PTE 被清空，可能会错误地获取原始页面
        if (!(*ptep & PTE_V)) // 页面被其他线程清除了
        {
            // 错误：直接获取原始只读页面
            // 正确做法应该是重新走 COW 流程
        }

        page_insert(mm->pgdir, new_page, ROUNDDOWN(addr, PGSIZE), perm);
    }
    return 0;
}
```

## 6.5 ucore COW 实现的安全性分析

当前实现的安全措施：

```
// kern/mm/vmm.c - 当前安全的实现
int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr)
{
    // 1. 整个函数是原子执行的（没有调度点）
    // 2. 从获取 PTE 到更新 PTE 是连续操作
    // 3. 没有中间状态可以被利用

    pte_t *ptep = get_pte(mm->pgdir, addr, 1);

    if ((*ptep & PTE_COW) && !(*ptep & PTE_W))
    {
        struct Page *old_page = pte2page(*ptep);
```

```

    if (page_ref(old_page) == 1)
    {
        // 原子操作：直接修改 PTE
        *ptep = (*ptep & ~PTE_COW) | PTE_W;
        tlb_invalidate(mm->pgdir, addr);
        return 0;
    }

    // 分配、复制、映射是连续的，没有竞态窗口
    struct Page *new_page = alloc_page();
    memcpy(page2kva(new_page), page2kva(old_page), PGSIZE);
    page_insert(mm->pgdir, new_page, ROUNDDOWN(addr, PGSIZE), perm);
}
return 0;
}

```

## 6.6 如果 ucore 将来支持多线程的防护措施

```

// 安全的多线程 COW 实现
int do_pgfault_safe(struct mm_struct *mm, uint32_t error_code, uintptr_t addr)
{
    int ret = 0;

    // 1. 获取 mm 锁，防止其他线程并发修改
    lock_mm(mm);

    pte_t *ptep = get_pte(mm->pgdir, addr, 1);
    if (ptep == NULL) {
        ret = -E_NO_MEM;
        goto out;
    }

    // 2. 在锁保护下检查和处理 COW
    if ((*ptep & PTE_COW) && !(*ptep & PTE_W))
    {
        // 2.1 保存当前 PTE 值用于一致性检查
        pte_t old_pte = *ptep;

        struct Page *old_page = pte2page(*ptep);

        if (page_ref(old_page) == 1)
        {
            // 原子更新
            *ptep = (*ptep & ~PTE_COW) | PTE_W;
            tlb_invalidate(mm->pgdir, addr);
            goto out;
        }

        struct Page *new_page = alloc_page();
        if (new_page == NULL) {
            ret = -E_NO_MEM;

```



```
        goto out;
    }

    // 2.2 复制前再次检查 PTE 是否被修改（防御性编程）
    if (*ptep != old_pte) {
        // PTE 被修改了，释放新页面，重试
        free_page(new_page);
        ret = -E_AGAIN; // 告诉调用者重试
        goto out;
    }

    memcpy(page2kva(new_page), page2kva(old_page), PGSIZE);

    // 2.3 原子性地更新页表
    uint32_t perm = (*ptep & PTE_USER & ~PTE_COW) | PTE_W;
    page_insert(mm->pgdir, new_page, ROUNDDOWN(addr, PGSIZE), perm);
}

out:
    // 3. 释放锁
    unlock_mm(mm);
    return ret;
}
```

关键防护原则：

| 原则     | 说明              | 实现方式              |
|--------|-----------------|-------------------|
| 锁保护    | 整个 COW 处理过程必须原子 | lock_mm/unlock_mm |
| 一致性检查  | 操作前后验证 PTE 未被修改 | 保存并比较 old_pte     |
| 最小竞态窗口 | 减少锁内的操作时间       | 优化代码路径            |
| 防御性编程  | 假设其他线程可能干扰      | 多次检查状态            |

## 6.7 Linux 的修复方案

Linux 通过引入 `FOLL_COW` 标志并检查 `pte_dirty()` 位来修复：

```
// Linux 修复后的逻辑
// commit 19be0eaffa3ac7d8eb6784ad9bdb7d67ed8e619

// 引入新的内部标志 FOLL_COW 来标记"已完成 COW"
// 而不是通过修改 FOLL_WRITE 来玩危险的游戏

// 使用 pte dirty 标志来验证 FOLL_COW 标志仍然有效
if ((flags & FOLL_COW) && !pte_dirty(pte)) {
    // COW 已经被其他操作破坏，需要重新处理
    return NULL;
}
```

## 6.8 总结

| 方面       | 说明                                |
|----------|-----------------------------------|
| 漏洞本质     | 多线程环境下 COW 处理的竞态条件                |
| 攻击方式     | madvise + /proc/self/mem 或 ptrace |
| ucore 风险 | 当前不存在（单线程、缺少关键系统调用）               |
| 未来风险     | 如果支持多线程需要特别注意锁保护                  |
| 防护核心     | 原子操作、一致性检查、锁保护                    |

## 7. 总结

本次 Challenge 成功实现了 COW 机制，主要工作包括：

- 1. 定义 PTE\_COW 标志位
- 2. 修改 fork 时的内存复制逻辑
- 3. 实现页面错误处理中的 COW 逻辑
- 4. 编写测试用例验证正确性

COW 机制显著优化了 fork 操作的性能，是现代操作系统的重要特性。同时，通过分析 Dirty COW 漏洞，我们认识到在实现 COW 时需要特别注意并发安全问题。